Flutter Mobile Tests

Root: E:\Y3S1\Software Engineering Framworks\project-demo\RentWiseAI

Terminal path: <repository root>\RentWise-Mobile
From repository root:
  cd RentWise-Mobile

1. Install dependencies:
  flutter pub get

2. Run all assessment unit/widget tests (mocked API; no emulator required):
  flutter test test/mobile_assessment --reporter expanded

3. Run one test group:
  flutter test test/mobile_assessment/auth_widgets_test.dart
  flutter test test/mobile_assessment/browse_widgets_test.dart
  flutter test test/mobile_assessment/details_session_test.dart

4. Run one test ID:
  flutter test test/mobile_assessment --plain-name MOB-13

5. Run all existing and assessment unit/widget tests (regression):
  flutter test --reporter expanded

6. Real backend integration setup:
  Start Android emulator / connect phone with USB debugging.
  flutter devices
  Copy-Item integration_test/mobile_assessment/live-config.example.json ‘
    integration_test/mobile_assessment/live-config.json
  Copy only if live-config.json does not already exist; preserve your configured file.
  Edit live-config.json: API_BASE_URL, E2E_EMAIL, E2E_PASSWORD, E2E_PROPERTY_ID.
  URL must end in /api. Use a Tenant test account and an active, unbooked listing
  visible on the first browse page. Start local backend if using a local URL.

7. Run real backend integration (replace DEVICE_ID from flutter devices):
  flutter test integration_test/mobile_assessment/live_backend_test.dart ‘
    -d DEVICE_ID ‘
    --dart-define-from-file=integration_test/mobile_assessment/live-config.json ‘
    --reporter expanded

All mobile coverage: run step 5 and step 7 separately.
The standard flutter test command does not include device integration tests.
For Android emulator, a PC-local backend uses 10.0.2.2, not localhost.

Backend API and Database Tests

Root: E:\Y3S1\Software Engineering Framworks\project-demo\RentWiseAI

Terminal path: <repository root>
If currently inside RentWise-Backend: cd ..

1. Prepare:
  Install .NET 8 SDK.
  Start Docker Desktop with Linux containers for API/database tests.
  Unit-only tests do not need Docker.

2. Restore packages:
  dotnet restore tests/BackendAssessment/BackendAssessment.Tests.csproj

3. Run all backend assessment tests (unit + API/security + database):
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj

4. Run unit tests only:
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "Area=Unit" --logger "console;verbosity=normal"

5. Run API integration and security tests:
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "Area=API|Area=Security" --logger "console;verbosity=normal"

6. Run database tests only:
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "Area=Database" --logger "console;verbosity=normal"

7. Retest one test ID:
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "FullyQualifiedName~API_08"
  dotnet test tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "FullyQualifiedName~DB_02"

API tests: in-process WebApplicationFactory + real services + disposable PostgreSQL.
DB-01: relationships. DB-02: NOT NULL / unique / FK constraints.
DB-03: actual migrations, transaction rollback and commit.
The database container is created and cleaned up automatically.
You do not manually start backend/frontend/Python services for these xUnit tests.

If running from RentWise-Backend instead, use:
  dotnet test ../tests/BackendAssessment/BackendAssessment.Tests.csproj ‘
    --filter "Area=Database"

Postman / Newman and Evidence Steps

Root: E:\Y3S1\Software Engineering Framworks\project-demo\RentWiseAI

Terminal path: <repository root>\tests\BackendAssessment\Newman
From repository root:
  cd tests/BackendAssessment/Newman

1. Install Newman:
  npm install

2. Set up environment (copy only if the local file does not exist):
  Copy-Item environment.example.json environment.local.json
  Edit baseUrl (ending in /api), tenantToken and ownerId.
  tenantToken: valid Tenant JWT, without the Bearer prefix.
  ownerId: test owner GUID. Use a dedicated test backend/account.
  A backend must be running/reachable; this does not use the xUnit container.

3. Run the full Newman security collection:
  New-Item -ItemType Directory -Force ../evidence
  npm test

Checks: API-11 unauthenticated request -> 401; valid Tenant profile;
API-12 wrong-role property creation -> 403.
Report: tests/BackendAssessment/evidence/newman-results.json

Postman option: import security.postman_collection.json and
environment.example.json; fill test values and run the collection.

Steps to follow for every suite:
1. Run initial tests; save console output and applicable TRX/JSON report.
2. Preserve initial evidence before another run overwrites it.
3. If a test fails, reproduce the exact ID and record it in DEFECTS.md.
4. Separate application defects from test/configuration errors.
5. Fix important defects; rerun the failing test and save retest evidence.
6. Run the full suite for regression; record the actual outcome.
7. Close a defect only after a passing retest.

Defect registers:
  RentWise-Mobile/test/mobile_assessment/DEFECTS.md
  tests/BackendAssessment/DEFECTS.md

Never commit live-config.json, environment.local.json or secrets/tokens.
All commands are PowerShell. A trailing backtick continues a command
on the next line; do not put spaces after the backtick.